UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO - UNIVESP
ENGENHARIA DA COMPUTAÇÃO

MAURÍCIO MATHEUS DIAS OLIVEIRA

ECOSSISTEMA ARGUS

ARQUITETURA IOT, EDGE COMPUTING E INTELIGÊNCIA ARTIFICIAL APLICADOS À TECNOLOGIA ASSISTIVA

SÃO PAULO - SP
2026

1 VISÃO GERAL DA ARQUITETURA

1.1 Mapa Mental Estrutural

Abaixo apresenta-se a estrutura hierárquica do Ecossistema ARGUS, dividida em seus três pilares fundamentais:

1 Hardware IoT (Infraestrutura Física)

- Microcontrolador ESP32 (Core Processamento)
- Leitor RFID RC522 (Barramento SPI)
- Feedback: Display OLED (I2C) e Áudio com Amplificador/Speaker Reciclado

2 Software e Mobile (Interface)

- App Android Nativo desenvolvido em Kotlin
- Dashboard Web Hospedada no ESP32 (HTML5, CSS3, JS)
- Comunicação Assíncrona via AJAX / Fetch API

3 Inteligência Artificial (Processamento Cognitivo)

- Visão Computacional no App Nativo via YOLOv8 (Obstáculos)
- Motor OCR via ML Kit para leitura de textos e livros em tempo real
- Integração de LLMs para Semântica e Assistente Médico

1.2 Fluxogramas de Execução

O fluxo de dados da arquitetura ocorre de maneira paralela. A seguir, ilustram-se as duas rotinas principais do ecossistema:

A) Fluxograma de Validação de Acesso (IoT Terminal):

```
[ Tag RFID ] → [ Leitor RC522 (SPI) ] → [ ESP32 Processa Identidade ] →
[ Fetch API (AJAX) ] → [ Dashboard & Feedback OLED/Áudio ]
```

B) Fluxograma de Assistência Visual (Edge Computing Mobile):

```
[ Câmera (Frames) ] → [ Inferência Offline (Smartphone) ] → [ YOLOv8
(Espaço) / OCR (Texto) ] → [ LLM (Semântica) ] → [ Voz (Motor TTS) ]
```

2 INTRODUÇÃO E PLATAFORMAS DE DESENVOLVIMENTO

O presente documento detalha a arquitetura de engenharia do Projeto ARGUS, um ecossistema de tecnologia assistiva focado em acessibilidade em ambientes de saúde. A construção do sistema exigiu a orquestração de múltiplas plataformas e linguagens de programação.

O firmware dos microcontroladores foi desenvolvido em linguagem C/C++ utilizando a Arduino IDE. A interface do aplicativo móvel nativo e os algoritmos de visão computacional foram construídos em Kotlin através do Android Studio. Para a construção da Dashboard Web hospedada no dispositivo IoT, utilizou-se HTML5, CSS3 e JavaScript assíncrono. Adicionalmente, o ecossistema faz forte uso de Inteligência Artificial, empregando Modelos de Linguagem de Grande Escala (LLM) e Reconhecimento Óptico de Caracteres (OCR).

3 ARQUITETURA DE HARDWARE (TERMINAL IOT E RFID)

3.1 O Microcontrolador ESP32 e Leitura RFID

O núcleo de segurança do ambiente clínico é gerenciado por um microcontrolador ESP32. A validação de identidade dos pacientes é realizada por um leitor RFID RC522 acoplado via barramento SPI. Ao detectar uma tag autorizada, o firmware realiza o cruzamento de dados na memória do microcontrolador e libera as credenciais, atualizando simultaneamente o status da rede local.

3.2 Feedback Sensorial: Áudio Sustentável e Display OLED

Para garantir acessibilidade física, o terminal possui feedback visual e auditivo. Um display OLED exibe animações dinâmicas de status. Simultaneamente, o sistema de áudio foi projetado com foco em sustentabilidade (ESG): utiliza-se um amplificador dedicado (via barramento I2S) conectado a um alto-falante reciclado de 8 Ohms. Essa montagem garante que a emissão dos bipes sonoros de validação seja clara e audível no ambiente clínico, reaproveitando lixo eletrônico.

4 ENGENHARIA DE SOFTWARE E INTELIGÊNCIA ARTIFICIAL

4.1 O Aplicativo Mobile ARGUS (Interface Híbrida)

Desenvolvido nativamente em Kotlin, o aplicativo atua como os 'olhos e a voz' do usuário. Seu design possui alto contraste para baixa visão, e a interface reage ativamente através de um sistema híbrido de processamento na borda (Edge Computing) e em nuvem.

4.2 Integração Mobile e Terminal RFID

A interface entre o hardware e o software ocorre através de uma arquitetura cliente-servidor embutida. O ESP32 hospeda uma dashboard web responsiva. Ao invés de recarregar a página, o código JavaScript utiliza Fetch API (AJAX) para consultar endpoints em tempo real. Quando o RFID valida um acesso, o JSON é injetado na DOM da aplicação de forma instantânea.

4.3 Visão Espacial e Leitura com IA: YOLOv8 e OCR no App

O aplicativo integra nativamente poderosos motores de inferência visual. Utilizando a câmera do dispositivo móvel, o modelo YOLOv8 rastreia obstáculos em tempo real (como pessoas, garrafas e móveis). Em conjunto, o app faz o uso intensivo da tecnologia OCR (Optical Character Recognition) via ML Kit. O usuário pode apontar a câmera do celular para capas de livros ou textos impressos, e a IA extrai o texto do mundo real, passando-o imediatamente para o motor Text-To-Speech (TTS), que lê as palavras em voz alta.

4.4 O Papel Analítico das LLMs (Large Language Models)

Para dotar o aplicativo de uma capacidade de compreensão semântica profunda, a arquitetura de software integra a tecnologia de LLMs (Large Language Models). Em vez de respostas programadas estáticas, o LLM permite que o aplicativo interprete intenções, processe os textos extraídos via OCR com maior inteligência conversacional e atue como um assistente médico cognitivo embarcado no dispositivo do usuário.

5 DESAFIOS TÉCNICOS E EVOLUÇÃO ARQUITETURAL

5.1 O Gargalo de Rede: ESP32-CAM e Servidor Python

O design final da arquitetura de processamento visual resultou de duas refatorações críticas baseadas em testes práticos. Inicialmente, o projeto utilizava um módulo ESP32-CAM para captura de vídeo, porém o hardware provou-se

ineficiente e extremamente lento para esse fim. Na segunda iteração, tentou-se utilizar a câmera do celular e um servidor externo em Python para processar a IA via rede WiFi. Contudo, essa abordagem também falhou: a latência (ping) do servidor atrasava a inferência de imagens, tornando o sistema perigoso para a locomoção de um deficiente visual.

5.2 A Solução: App Nativo e Edge Computing

Para resolver definitivamente o problema de latência, abandonou-se o servidor Python e adotou-se o desenvolvimento de um Aplicativo Nativo em Android (Kotlin). A rede foi descartada do processo de visão em favor do Edge Computing (Computação de Borda): todo o peso da Inteligência Artificial (YOLOv8 e OCR) passou a ser processado localmente pelo hardware do próprio smartphone, garantindo uma taxa de atualização instantânea e uso offline da visão assistiva.

6 DEMONSTRAÇÃO PRÁTICA E VALIDAÇÃO

Para comprovar a eficiência do ecossistema ARGUS, a validação do projeto foi dividida em duas demonstrações práticas em vídeo. O primeiro vídeo detalha as funcionalidades de Inteligência Artificial do aplicativo mobile (leitura via OCR e Visão Espacial processadas de forma nativa). O segundo vídeo comprova a comunicação física do hardware IoT, ilustrando a leitura da Tag RFID no ESP32, o acionamento do amplificador e do alto-falante, e a atualização instantânea da Dashboard Web em tempo real.

1. **Vídeo Mobile (OCR, LLM e Visão Edge):** https://www.youtube.com/watch?v=AZ03ZFNZcRM

2. **Vídeo Hardware (RFID, Som e Web):** https://www.youtube.com/watch?v=H1biLwCZ6vE&t=3s

3. Download (App Argus): https://www.mediafire.com/file/hdiionlcu9gl2uo/ArgusAPP.apk/file

7 CONSIDERAÇÕES FINAIS

O Ecossistema ARGUS ultrapassa a barreira do conceito acadêmico e consolida-se como um produto de Engenharia da Computação viável. Ao superar limitações de hardware (como a lentidão da ESP32-CAM), eliminar os gargalos de servidores remotos em Python transferindo a IA para a borda no App Nativo, e integrar soluções sustentáveis na camada física, o projeto demonstra uma arquitetura responsiva, segura e profundamente focada na autonomia humana.